# Pemodelan Data Sistem Smart Agriculture

Pemodelan Konseptual, Logis, dan Fisik (ERD, Normalisasi 1NF - 3NF, dan SQL DDL) — Pembaruan: Ambang Batas Menyesuaikan Tanaman

**Latar belakang.** Sistem Smart Agriculture memantau kondisi lahan (kelembaban tanah, suhu udara, dan pH tanah) melalui sensor yang dipasang pada setiap zona lahan. Pada rancangan sebelumnya, ambang batas normal (nilai_min dan nilai_max) disimpan di master JENIS_SENSOR sehingga berlaku seragam untuk seluruh zona dan seluruh jenis tanaman. Tantangan bisnis yang muncul adalah usaha berkembang dari satu jenis tanaman menjadi beberapa tanaman, sedangkan batas normal kelembaban, suhu, dan pH berbeda untuk setiap tanaman. Dengan standar tunggal per jenis sensor, kondisi yang normal bagi satu tanaman dapat dinilai abnormal bagi tanaman lain (dan sebaliknya), sehingga peringatan menjadi tidak akurat. Kebutuhan ini menambah proses bisnis baru, yaitu **penilaian hasil pengukuran memakai ambang batas yang sesuai dengan tanaman yang sedang ditanam di zona tersebut**. Kebutuhan ini diwujudkan dengan menambahkan entitas **TANAMAN** dan **AMBANG_BATAS**, serta mengubah struktur ZONA_LAHAN, JENIS_SENSOR, dan PERINGATAN.

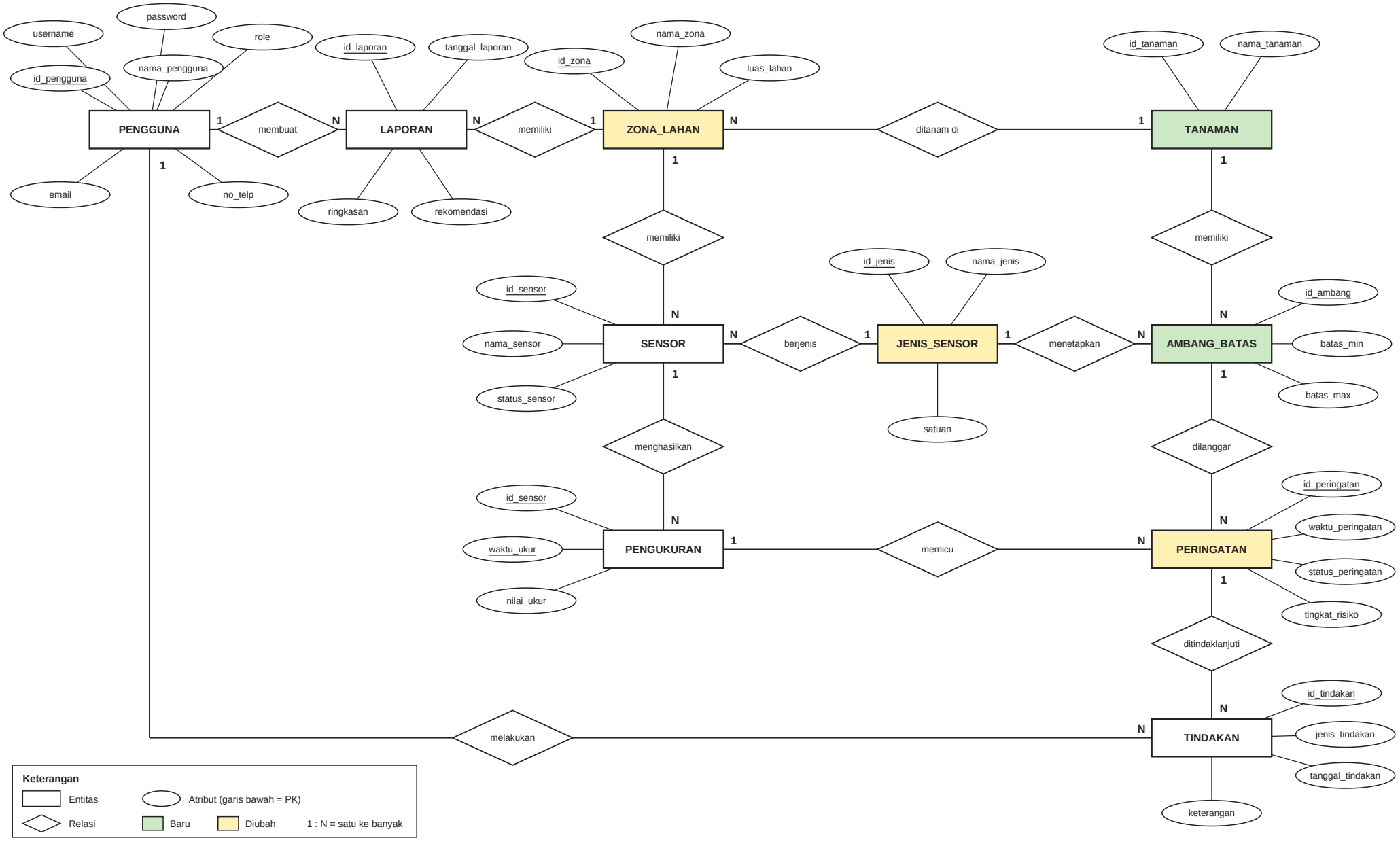
## Ringkasan perubahan terhadap model sebelumnya

| Entitas / Tabel | Status | Perubahan |
|---|---|---|
| TANAMAN | **[BARU]** | Master tanaman yang dibudidayakan (id_tanaman, nama_tanaman). |
| AMBANG_BATAS | **[BARU]** | Batas bawah dan atas yang ditetapkan per kombinasi tanaman dan jenis sensor. |
| JENIS_SENSOR | **[DIUBAH]** | Atribut nilai_min dan nilai_max dihapus karena dipindahkan ke AMBANG_BATAS; satuan tetap di sini. |
| ZONA_LAHAN | **[DIUBAH]** | Atribut jenis_tanaman (teks bebas) diganti id_tanaman sebagai foreign key ke TANAMAN. |
| PERINGATAN | **[DIUBAH]** | Ditambah id_ambang (foreign key) agar setiap peringatan dapat ditelusuri ke aturan yang dilanggar. |
| PENGGUNA, LAPORAN, SENSOR, PENGUKURAN, TINDAKAN | Tetap | Tidak berubah. |

## Asumsi dan batasan rancangan

- Satu zona lahan menanam **satu tanaman pada satu waktu**. Riwayat pergantian tanaman per musim tidak dimodelkan pada pembaruan ini.
- Ambang batas ditetapkan per kombinasi tanaman dan jenis sensor, dan berlaku sama untuk seluruh fase pertumbuhan.
- Nilai ambang pada contoh data adalah **contoh rancangan untuk demonstrasi**, bukan standar agronomis baku.
- Tabel rekap_dasbor_zona tidak dimodelkan sebagai entitas karena hanya berisi data turunan untuk tampilan dashboard.

# 1. Pemodelan Konseptual - Entity-Relationship Diagram (ERD)




*Gambar 1. ERD notasi Chen sistem Smart Agriculture (setelah penambahan entitas TANAMAN dan AMBANG_BATAS)*

## Penjelasan Gambar 1

ERD memetakan sepuluh entitas, atribut, dan aturan bisnis sistem. Persegi panjang adalah entitas, belah ketupat adalah relasi, dan elips adalah atribut. Atribut bergaris bawah adalah primary key atau bagian dari key. Angka 1 dan N pada garis menunjukkan kardinalitas relasi. Entitas berwarna hijau adalah entitas baru, sedangkan entitas berwarna kuning adalah entitas yang strukturnya diubah.

## a. Entitas dan atribut

| Entitas | Atribut pada ERD | Primary key | Keterangan |
|---|---|---|---|
| PENGGUNA | id_pengguna, nama_pengguna, username, password, role, email, no_telp | id_pengguna | Pengguna sistem dengan peran tertentu. |
| ZONA_LAHAN **[DIUBAH]** | id_zona, nama_zona, luas_lahan | id_zona | Area lahan yang dipantau. Tanaman yang ditanam diwakili relasi ke TANAMAN (atribut jenis_tanaman dihapus). |
| TANAMAN **[BARU]** | id_tanaman, nama_tanaman | id_tanaman | Master tanaman yang dibudidayakan; nama tanaman tersimpan satu kali. |
| JENIS_SENSOR **[DIUBAH]** | id_jenis, nama_jenis, satuan | id_jenis | Master jenis sensor beserta satuannya. nilai_min dan nilai_max dihapus karena diwakili AMBANG_BATAS. |
| AMBANG_BATAS **[BARU]** | id_ambang, batas_min, batas_max | id_ambang (unik: id_tanaman + id_jenis) | Batas normal pengukuran untuk satu jenis sensor pada satu tanaman. |
| SENSOR | id_sensor, nama_sensor, status_sensor | id_sensor | Perangkat pemantau; berada di satu zona dan memiliki satu jenis. |
| PENGUKURAN | id_sensor, waktu_ukur, nilai_ukur | id_sensor + waktu_ukur (komposit) | Hasil baca sensor pada waktu tertentu. |
| PERINGATAN **[DIUBAH]** | id_peringatan, waktu_peringatan, status_peringatan, tingkat_risiko | id_peringatan | Notifikasi dari pengukuran tertentu. id_sensor dan waktu_ukur menunjuk pengukuran pemicu; id_ambang menunjuk aturan yang dilanggar. |
| TINDAKAN | id_tindakan, jenis_tindakan, tanggal_tindakan, keterangan | id_tindakan | Penanganan atas suatu peringatan. |
| LAPORAN | id_laporan, tanggal_laporan, ringkasan, rekomendasi | id_laporan | Laporan kondisi zona yang dibuat pengguna. |

## b. Relasi dan kardinalitas

| No | Relasi | Entitas | Kardinalitas | Makna |
|---|---|---|---|---|
| 1 | membuat | PENGGUNA - LAPORAN | 1 : N | Satu pengguna dapat membuat banyak laporan. |
| 2 | memiliki | ZONA_LAHAN - LAPORAN | 1 : N | Satu zona memiliki banyak laporan. |
| 3 | memiliki | ZONA_LAHAN - SENSOR | 1 : N | Satu zona memiliki banyak sensor; setiap sensor berada di satu zona. |
| 4 | ditanam di **[BARU]** | TANAMAN - ZONA_LAHAN | 1 : N | Satu tanaman dapat ditanam di banyak zona; setiap zona menanam satu tanaman pada satu waktu (boleh belum ditanami). |
| 5 | berjenis | JENIS_SENSOR - SENSOR | 1 : N | Satu jenis dipakai banyak sensor; setiap sensor memiliki tepat satu jenis. |
| 6 | memiliki **[BARU]** | TANAMAN - AMBANG_BATAS | 1 : N | Satu tanaman memiliki banyak ambang, satu untuk setiap jenis sensor. |
| 7 | menetapkan **[BARU]** | JENIS_SENSOR - AMBANG_BATAS | 1 : N | Satu jenis sensor memiliki ambang yang berbeda untuk tiap tanaman. |
| 8 | menghasilkan | SENSOR - PENGUKURAN | 1 : N | Satu sensor menghasilkan banyak pengukuran. |
| 9 | memicu | PENGUKURAN - PERINGATAN | 1 : N | Satu pengukuran dapat memicu peringatan. |
| 10 | dilanggar **[BARU]** | AMBANG_BATAS - PERINGATAN | 1 : N | Satu ambang dapat dilanggar pada banyak peringatan; setiap peringatan merujuk satu ambang. |
| 11 | ditindaklanjuti | PERINGATAN - TINDAKAN | 1 : N | Satu peringatan dapat ditindaklanjuti dengan banyak tindakan. |
| 12 | melakukan | PENGGUNA - TINDAKAN | 1 : N | Satu pengguna dapat melakukan banyak tindakan. |

### c. Aturan bisnis yang dipetakan

- Setiap sensor terpasang pada tepat satu zona lahan dan memiliki tepat satu jenis sensor; satuan ditetapkan per jenis sensor.
- Setiap zona lahan menanam paling banyak satu tanaman pada satu waktu, dan satu tanaman dapat ditanam di banyak zona.
- **Ambang batas (batas_min dan batas_max) ditetapkan per kombinasi tanaman dan jenis sensor**; kombinasi tersebut unik dan batas_min harus lebih kecil daripada batas_max.
- Sebuah pengukuran dinilai terhadap ambang batas milik tanaman yang ditanam di zona sensor tersebut. Pengukuran yang berada di luar ambang dapat memicu peringatan, dan peringatan mencatat ambang yang dilanggar.
- Peringatan ditindaklanjuti melalui tindakan yang dilakukan pengguna, dan pengguna membuat laporan per zona berisi ringkasan dan rekomendasi.
- Asumsi lama bahwa ambang batas berlaku seragam untuk seluruh zona dan jenis tanaman **tidak lagi berlaku** dan digantikan aturan per tanaman di atas.

## 2. Pemodelan Logis - Skema Relasional dan Normalisasi

### 2.1 Transformasi ERD ke skema relasional

Aturan transformasi yang dipakai: setiap entitas menjadi satu relasi (tabel); atribut menjadi kolom; relasi 1 : N diwujudkan dengan menaruh primary key sisi 1 sebagai foreign key di sisi N; entitas dengan key gabungan memakai primary key komposit. Pada skema di bawah, garis bawah menandai primary key dan tanda bintang (*) menandai foreign key.

### Skema relasional

zona_lahan (id_zona, nama_zona, luas_lahan, id_tanaman*) **[DIUBAH]**
tanaman (id_tanaman, nama_tanaman) **[BARU]**
pengguna (id_pengguna, nama_pengguna, username, password, role, email, no_telp)
jenis_sensor (id_jenis, nama_jenis, satuan) **[DIUBAH]**
ambang_batas (id_ambang, id_tanaman*, id_jenis*, batas_min, batas_max)  unik (id_tanaman, id_jenis) **[BARU]**
sensor (id_sensor, id_zona*, id_jenis*, nama_sensor, status_sensor)
pengukuran (id_sensor*, waktu_ukur, nilai_ukur)
peringatan (id_peringatan, id_sensor*, waktu_ukur*, id_ambang*, waktu_peringatan, status_peringatan, tingkat_risiko) **[DIUBAH]**
tindakan (id_tindakan, id_peringatan*, id_pengguna*, jenis_tindakan, tanggal_tindakan, keterangan)
laporan (id_laporan, id_zona*, id_pengguna*, tanggal_laporan, ringkasan, rekomendasi)

Catatan: id_tanaman pada zona_lahan dan id_ambang pada peringatan boleh kosong (NULL) agar zona yang belum ditanami dan peringatan lama (sebelum pembaruan) tetap dapat tersimpan.

## 2.2 Normalisasi 1NF dan 2NF

**Struktur awal yang dinormalisasi.** Tabel zona_lahan dan jenis_sensor versi lama, serta bentuk yang muncul bila kebutuhan ambang per tanaman ditambahkan langsung ke tabel jenis_sensor, yaitu relasi *jenis_sensor_ambang* (id_jenis, jenis_tanaman, nama_jenis, satuan, batas_min, batas_max) dengan primary key gabungan (id_jenis, jenis_tanaman).

**zona_lahan (sebelum)**

| id_zona (PK) | nama_zona | luas_lahan | jenis_tanaman |
|---|---|---|---|
| 1 | Zona A | 2,50 | Padi |
| 2 | Zona B | 1,80 | Cabai |
| 3 | Zona C | 3,20 | Padi |

**jenis_sensor_ambang (bentuk awal kebutuhan baru)**

| id_jenis (PK) | jenis_tanaman (PK) | nama_jenis | satuan | batas_min | batas_max |
|---|---|---|---|---|---|
| 1 | Padi | Kelembaban Tanah | % | 60,00 | 85,00 |
| 1 | Cabai | Kelembaban Tanah | % | 50,00 | 70,00 |
| 2 | Padi | Suhu Udara | °C | 24,00 | 32,00 |
| 2 | Cabai | Suhu Udara | °C | 22,00 | 30,00 |
| 3 | Padi | pH Tanah | pH | 5,50 | 7,00 |
| 3 | Cabai | pH Tanah | pH | 6,00 | 7,00 |

*Gambar 2. Tabel sebelum pemisahan (data contoh ilustrasi) dan analisis ketergantungan fungsional (2NF)*

## Penjelasan Gambar 2 - Bentuk Normal Pertama (1NF)

1NF mensyaratkan setiap kolom bernilai atomik, tidak ada grup kolom berulang, dan setiap baris dapat dibedakan lewat primary key.

| Kriteria 1NF | Hasil | Bukti pada tabel |
|---|---|---|
| Nilai atomik | Terpenuhi | Setiap sel hanya berisi satu nilai, misalnya jenis_tanaman berisi 'Padi', bukan daftar tanaman. |
| Memiliki primary key | Terpenuhi | id_zona unik pada zona_lahan; pasangan (id_jenis, jenis_tanaman) unik untuk setiap baris pada jenis_sensor_ambang. |
| Tidak ada grup berulang | Terpenuhi | Tidak ada kolom seperti tanaman1, tanaman2; setiap tanaman menjadi baris tersendiri. |

**Kesimpulan:** kedua tabel sudah memenuhi 1NF sehingga tidak ada perubahan struktur pada tahap ini. Catatan: kolom jenis_tanaman berupa teks bebas sehingga tidak ada yang mencegah penulisan tanaman yang sama dengan ejaan berbeda; hal ini ditangani pada tahap 3NF.

## Penjelasan Gambar 2 - Bentuk Normal Kedua (2NF)

2NF mensyaratkan tabel sudah 1NF dan setiap atribut non-key bergantung pada **seluruh** primary key (tidak ada ketergantungan parsial). Analisis ketergantungan fungsional pada jenis_sensor_ambang:

| Ketergantungan | Jenis | Keterangan |
|---|---|---|
| (id_jenis, jenis_tanaman) → batas_min | Penuh | Batas bawah ditentukan oleh kombinasi jenis sensor dan tanaman. |
| (id_jenis, jenis_tanaman) → batas_max | Penuh | Batas atas ditentukan oleh kombinasi jenis sensor dan tanaman. |
| id_jenis → nama_jenis | **Parsial** | Nama jenis hanya ditentukan oleh sebagian key (id_jenis); melanggar 2NF. |
| id_jenis → satuan | **Parsial** | Satuan hanya ditentukan oleh jenis sensor; '%' tertulis berulang pada baris Padi dan Cabai. |

**Kesimpulan:** primary key terdiri dari dua kolom dan terdapat ketergantungan parsial, sehingga jenis_sensor_ambang **belum memenuhi 2NF**. Akibatnya terjadi redundansi (nama dan satuan jenis berulang untuk setiap tanaman) dan risiko anomali pembaruan. Penyelesaiannya adalah memisahkan atribut yang bergantung pada id_jenis saja ke tabel jenis_sensor, dan menyisakan batas_min serta batas_max pada tabel ambang_batas yang berkunci kombinasi tanaman dan jenis sensor.

## 2.3 Normalisasi 3NF

**1. jenis_sensor**

| id_jenis (PK) | nama_jenis | satuan |
|---|---|---|
| 1 | Kelembaban Tanah | % |
| 2 | Suhu Udara | °C |
| 3 | pH Tanah | pH |

**2. tanaman (baru)**

| id_tanaman (PK) | nama_tanaman |
|---|---|
| 1 | Padi |
| 2 | Cabai |

**3. ambang_batas (baru)**

| id_ambang (PK) | id_tanaman (FK) | id_jenis (FK) | batas_min | batas_max |
|---|---|---|---|---|
| 1 | 1 | 1 | 60,00 | 85,00 |
| 2 | 1 | 2 | 24,00 | 32,00 |
| 3 | 1 | 3 | 5,50 | 7,00 |
| 4 | 2 | 1 | 50,00 | 70,00 |
| 5 | 2 | 2 | 22,00 | 30,00 |
| 6 | 2 | 3 | 6,00 | 7,00 |

**4. zona_lahan (struktur baru)**

| id_zona (PK) | nama_zona | luas_lahan | id_tanaman (FK) |
|---|---|---|---|
| 1 | Zona A | 2,50 | 1 |
| 2 | Zona B | 1,80 | 2 |
| 3 | Zona C | 3,20 | 1 |

*Gambar 3. Hasil pemisahan tabel menjadi jenis_sensor, tanaman, ambang_batas, dan zona_lahan (3NF)*

## Penjelasan Gambar 3 - Bentuk Normal Ketiga (3NF)

3NF mensyaratkan tabel sudah 2NF dan tidak ada ketergantungan transitif, yaitu atribut non-key tidak boleh ditentukan oleh atribut non-key lain.

- **Hasil 2NF:** jenis_sensor (id_jenis, nama_jenis, satuan) dan ambang_batas (id_jenis, jenis_tanaman, batas_min, batas_max) sudah bebas dari ketergantungan parsial. Bersama zona_lahan (id_zona, nama_zona, luas_lahan, jenis_tanaman), ketiganya diperiksa terhadap ketergantungan transitif; **tidak ditemukan** ketergantungan transitif, sehingga secara formal ketiganya sudah memenuhi 3NF.
- **Masalah yang tersisa (integritas):** jenis_tanaman berupa teks yang menjadi penghubung antara zona_lahan dan ambang_batas tanpa relasi induk. Salah ketik atau ejaan berbeda (misalnya 'Cabe' dan 'Cabai') akan membuat zona tidak menemukan ambangnya, dan nama tanaman tersimpan berulang di dua tabel.
- **Solusi:** dibuat master tanaman (id_tanaman, nama_tanaman). Tabel zona_lahan dan ambang_batas merujuknya lewat foreign key id_tanaman (tabel kedua sampai keempat pada Gambar 3). Jika kelak TANAMAN mendapat atribut lain (misalnya kategori atau umur panen), tidak terbentuk ketergantungan transitif id_zona → jenis_tanaman → atribut tanaman di dalam zona_lahan.

| Tabel | Ketergantungan fungsional setelah pemisahan | Hasil |
|---|---|---|
| jenis_sensor | id_jenis → nama_jenis, satuan | Semua langsung ke primary key |
| tanaman | id_tanaman → nama_tanaman | Semua langsung ke primary key |
| ambang_batas | id_ambang → id_tanaman, id_jenis, batas_min, batas_max; (id_tanaman, id_jenis) → id_ambang, batas_min, batas_max (kunci kandidat) | Semua bergantung pada key; tidak ada transitif |
| zona_lahan | id_zona → nama_zona, luas_lahan, id_tanaman | Semua langsung ke primary key |

**Kesimpulan:** ketergantungan parsial telah dihilangkan dan setiap atribut non-key hanya bergantung pada primary key tabelnya sehingga 3NF terpenuhi. Relasi baru yang terbentuk adalah TANAMAN (1) ke ZONA_LAHAN (N), TANAMAN (1) ke AMBANG_BATAS (N), dan JENIS_SENSOR (1) ke AMBANG_BATAS (N). Asumsi: nilai ambang pada gambar adalah contoh rancangan, bukan standar baku.

## 2.4 Pemeriksaan seluruh tabel

| Tabel | Primary key | 1NF | 2NF | 3NF | Catatan |
|---|---|---|---|---|---|
| zona_lahan | id_zona | Ya | Ya | Ya | jenis_tanaman diganti id_tanaman (FK). |
| tanaman | id_tanaman | Ya | Ya | Ya | Tabel baru hasil pemisahan. |
| jenis_sensor | id_jenis | Ya | Ya | Ya | nilai_min dan nilai_max dipindah ke ambang_batas. |
| ambang_batas | id_ambang (unik: id_tanaman + id_jenis) | Ya | Ya | Ya | Tabel baru; batas bergantung pada kombinasi tanaman dan jenis. |
| sensor | id_sensor | Ya | Ya | Ya | Tidak berubah. |
| pengukuran | id_sensor + waktu_ukur | Ya | Ya | Ya | nilai_ukur bergantung pada kedua kolom key. |
| peringatan | id_peringatan | Ya | Ya | Ya | Ditambah id_ambang (FK). |
| tindakan | id_tindakan | Ya | Ya | Ya | jenis_tindakan masih teks bebas (di luar lingkup pembaruan ini). |
| pengguna | id_pengguna | Ya | Ya | Ya | Kolom password sebaiknya berisi hash. |
| laporan | id_laporan | Ya | Ya | Ya | |
| rekap_dasbor_zona | id_zona | Ya | Ya | Perlu ditinjau | Menyimpan data turunan (redundan); disarankan menjadi VIEW atau dijelaskan sebagai denormalisasi untuk performa dashboard. |

# 3. Pemodelan Fisik - SQL DDL

Skema relasional diterjemahkan menjadi struktur tabel MySQL (InnoDB) lengkap dengan primary key, foreign key, dan constraint validasi. Berikut kode untuk dua tabel yang baru dan perubahan pada tiga tabel lama (zona_lahan, jenis_sensor, peringatan) akibat penambahan proses bisnis; tabel lainnya tetap seperti semula.

## 3.1 Tabel tanaman

```
CREATE TABLE `tanaman` (
  `id_tanaman`   int NOT NULL AUTO_INCREMENT,
  `nama_tanaman` varchar(100) NOT NULL,
  PRIMARY KEY (`id_tanaman`),
  UNIQUE KEY `uk_nama_tanaman` (`nama_tanaman`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

**Penjelasan:** id_tanaman adalah primary key yang bertambah otomatis. nama_tanaman wajib diisi (NOT NULL) dan unik (UNIQUE KEY) agar satu tanaman tidak terdaftar ganda.

## 3.2 Tabel ambang_batas

```
CREATE TABLE `ambang_batas` (
  `id_ambang`  int NOT NULL AUTO_INCREMENT,
  `id_tanaman` int NOT NULL,
  `id_jenis`   int NOT NULL,
  `batas_min`  decimal(10,2) NOT NULL,
  `batas_max`  decimal(10,2) NOT NULL,
  PRIMARY KEY (`id_ambang`),
  UNIQUE KEY `uk_tanaman_jenis` (`id_tanaman`, `id_jenis`),
  KEY `fk_ambang_jenis` (`id_jenis`),
  CONSTRAINT `fk_ambang_tanaman` FOREIGN KEY (`id_tanaman`)
    REFERENCES `tanaman` (`id_tanaman`) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT `fk_ambang_jenis` FOREIGN KEY (`id_jenis`)
    REFERENCES `jenis_sensor` (`id_jenis`) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT `chk_ambang_min_max` CHECK (`batas_min` < `batas_max`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

**Penjelasan:** primary key id_ambang; pasangan (id_tanaman, id_jenis) dibuat unik sehingga setiap tanaman hanya punya satu ambang untuk tiap jenis sensor. Tipe decimal(10,2) dipilih agar nilai ambang akurat sampai dua desimal (misalnya pH 5,50). Constraint chk_ambang_min_max menolak data yang batas minimumnya tidak lebih kecil daripada batas maksimumnya. ON DELETE RESTRICT mencegah penghapusan tanaman atau jenis sensor yang masih memiliki ambang, dan ON UPDATE CASCADE meneruskan perubahan key.

## 3.3 Data awal

```
INSERT INTO `tanaman` (`id_tanaman`, `nama_tanaman`) VALUES
(1, 'Padi'), (2, 'Cabai');

INSERT INTO `ambang_batas` (`id_ambang`, `id_tanaman`, `id_jenis`, `batas_min`, `batas_max`) VALUES
(1, 1, 1, 60.00, 85.00), (2, 1, 2, 24.00, 32.00), (3, 1, 3, 5.50, 7.00),
(4, 2, 1, 50.00, 70.00), (5, 2, 2, 22.00, 30.00), (6, 2, 3, 6.00, 7.00);

UPDATE `zona_lahan` SET `id_tanaman` = 1 WHERE `id_zona` IN (1, 3);
UPDATE `zona_lahan` SET `id_tanaman` = 2 WHERE `id_zona` = 2;
```

**Penjelasan:** data ini sama dengan yang ditampilkan pada Gambar 3. Tabel tanaman harus diisi lebih dahulu karena ambang_batas dan zona_lahan memiliki foreign key ke tabel tersebut. Nilai ambang adalah contoh ilustrasi, bukan acuan agronomis.

## 3.4 Migrasi pada database yang sudah terisi

```
-- 1. Buat tanaman dari teks lama (dibersihkan spasi)
INSERT INTO `tanaman` (`nama_tanaman`)
SELECT DISTINCT TRIM(`jenis_tanaman`) FROM `zona_lahan`
WHERE `jenis_tanaman` IS NOT NULL AND TRIM(`jenis_tanaman`) <> '';

-- 2. (Buat tabel ambang_batas seperti 3.2), lalu salin batas lama sebagai nilai awal tiap tanaman
INSERT INTO `ambang_batas` (`id_tanaman`, `id_jenis`, `batas_min`, `batas_max`)
SELECT t.`id_tanaman`, js.`id_jenis`, js.`nilai_min`, js.`nilai_max`
FROM `tanaman` t CROSS JOIN `jenis_sensor` js;

-- 3. zona_lahan: ganti teks bebas dengan foreign key
ALTER TABLE `zona_lahan` ADD COLUMN `id_tanaman` int NULL;
UPDATE `zona_lahan` z
JOIN `tanaman` t ON t.`nama_tanaman` = TRIM(z.`jenis_tanaman`)
SET z.`id_tanaman` = t.`id_tanaman`;
ALTER TABLE `zona_lahan`
  ADD CONSTRAINT `fk_zona_tanaman` FOREIGN KEY (`id_tanaman`)
    REFERENCES `tanaman` (`id_tanaman`) ON DELETE RESTRICT ON UPDATE CASCADE;

-- 4. peringatan: catat aturan yang dilanggar
ALTER TABLE `peringatan` ADD COLUMN `id_ambang` int NULL,
  ADD CONSTRAINT `fk_peringatan_ambang` FOREIGN KEY (`id_ambang`)
    REFERENCES `ambang_batas` (`id_ambang`) ON DELETE RESTRICT ON UPDATE CASCADE;
UPDATE `peringatan` p
JOIN `sensor` s       ON s.`id_sensor` = p.`id_sensor`
JOIN `zona_lahan` z   ON z.`id_zona` = s.`id_zona`
JOIN `ambang_batas` a ON a.`id_tanaman` = z.`id_tanaman` AND a.`id_jenis` = s.`id_jenis`
SET p.`id_ambang` = a.`id_ambang`;

-- 5. Pembersihan kolom lama (jalankan HANYA setelah hasil langkah 1-4 diverifikasi).
--    Constraint CHECK lama harus dilepas dulu karena merujuk kolom yang akan dihapus.
ALTER TABLE `jenis_sensor` DROP CONSTRAINT `chk_ambang_batas`;
ALTER TABLE `jenis_sensor` DROP COLUMN `nilai_min`, DROP COLUMN `nilai_max`;
ALTER TABLE `zona_lahan`   DROP COLUMN `jenis_tanaman`;
```

**Penjelasan:** karena tabel lain (sensor, pengukuran, peringatan) memiliki foreign key ke tabel yang berubah, tabel tidak dibuat ulang melainkan diubah bertahap. Urutannya: (1) bentuk master tanaman dari teks lama; (2) salin batas lama dari jenis_sensor sebagai nilai awal ambang tiap tanaman, lalu sesuaikan per tanaman; (3) tambah id_tanaman pada zona_lahan, isi dengan memetakan nama tanaman lama ke master, lalu pasang foreign key; (4) tambah id_ambang pada peringatan dan isi dari tanaman zona terkait; (5) setelah diverifikasi, lepas constraint CHECK lama dan hapus kolom yang sudah tidak dipakai. Baris peringatan lama yang tidak menemukan pasangan ambang tetap bernilai NULL.

## 3.5 Ringkasan constraint

| Jenis constraint | Penerapan | Tujuan |
|---|---|---|
| PRIMARY KEY | tanaman.id_tanaman, ambang_batas.id_ambang | Identitas unik setiap baris. |
| UNIQUE | tanaman.nama_tanaman; ambang_batas (id_tanaman, id_jenis) | Mencegah tanaman ganda dan ambang ganda untuk kombinasi yang sama. |
| FOREIGN KEY | ambang_batas.id_tanaman → tanaman; ambang_batas.id_jenis → jenis_sensor; zona_lahan.id_tanaman → tanaman; peringatan.id_ambang → ambang_batas | Menjaga integritas referensial antar tabel. |
| CHECK | batas_min < batas_max | Validasi isi data. |
| NOT NULL | Seluruh kolom pada tanaman dan ambang_batas; zona_lahan.id_tanaman dan peringatan.id_ambang sengaja boleh NULL | Data wajib lengkap; NULL hanya pada zona yang belum ditanami dan peringatan lama. |
| ON DELETE RESTRICT / ON UPDATE CASCADE | Keempat foreign key di atas | Mencegah penghapusan induk yang masih dipakai dan menyebarkan perubahan key. |